PUC Minas

LAB02	Laboratório 02 - 20 pontos

INFORMAÇÕES DOCENTE						
CURSO: ENGENHARIA DE SOFTWARE	DISCIPLINA: LABORATÓRIO DE EXPERIMENTAÇÃO DE SOFTWARE	TURNO	MANHÃ	TARDE	NOITE	PERÍODO/SALA: 6º
					x	
ALUNOS (A): Leonardo Vieira Machado e Nicolas Almeida Prado da Silva						

Características de qualidade de sistemas java

1. Introdução

No processo de desenvolvimento de sistemas *open-source*, onde diversos desenvolvedores colaboram em partes distintas do código, existe um risco inerente à evolução dos atributos de qualidade interna do software. Ao adotar essa abordagem colaborativa, aspectos fundamentais como modularidade, manutenibilidade e legibilidade podem se tornar vulneráveis.

Nesse contexto, o objetivo deste estudo é analisar aspectos de qualidade de repositórios desenvolvidos na linguagem Java, correlacionando essas características de qualidade com o processo de desenvolvimento dos mesmos. Essa análise é feita sob a perspectiva de métricas de produto, calculadas por meio da ferramenta de análise estática CK.

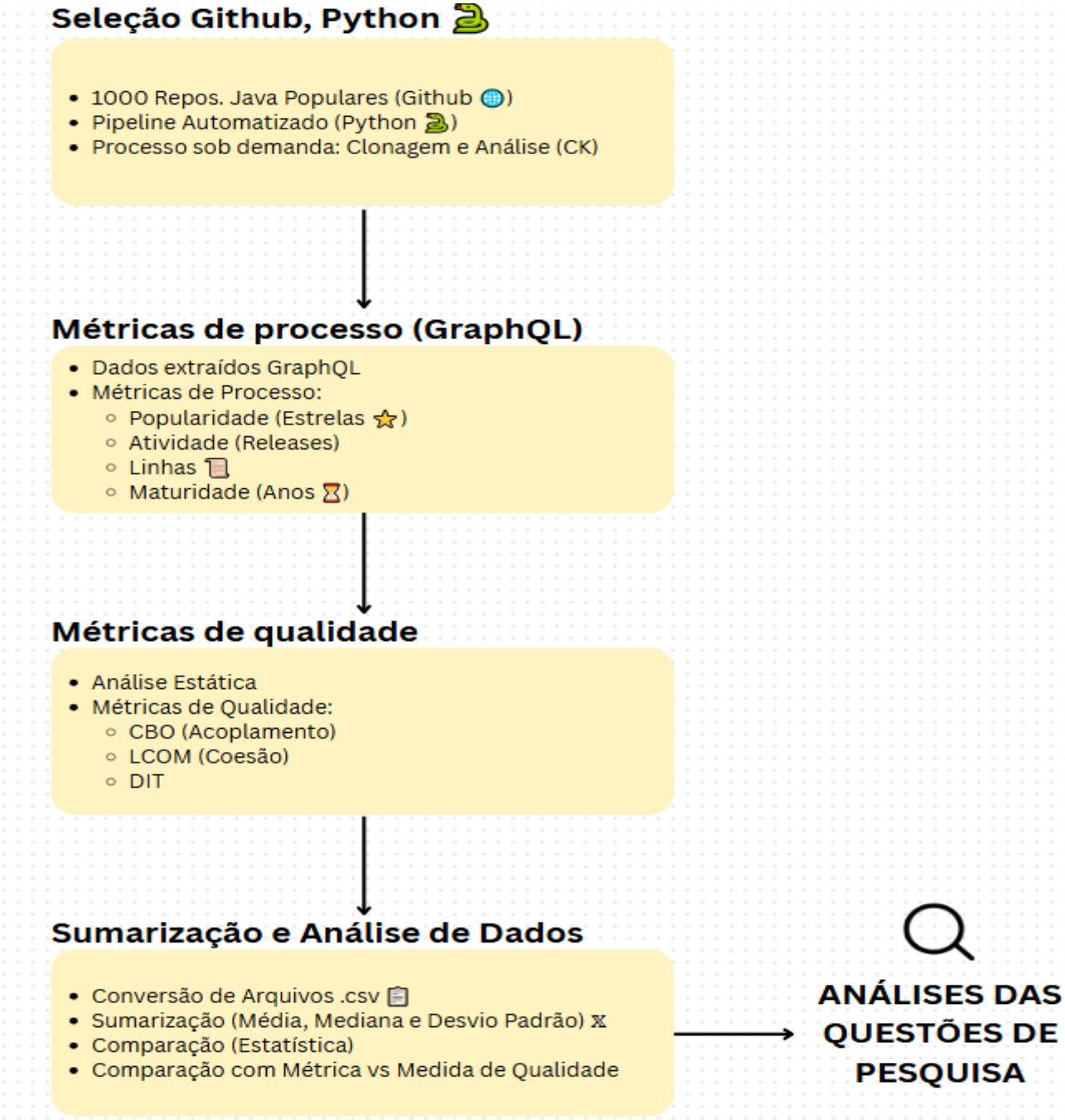
Para guiar o estudo, foram levantadas as seguintes questões de pesquisa (RQs) e suas respectivas hipóteses informais:

- **RQ 01:** Qual a relação entre a popularidade dos repositórios e as suas características de qualidade?
    - *Hipótese:* Espera-se que repositórios altamente populares (com muitas estrelas) apresentem métricas de qualidade melhores (menor acoplamento e maior coesão), uma vez que possuem comunidades maiores revisando o código e exigindo padrões mais rígidos de aceitação (via ferramentas de CI/CD).
- **RQ 02:** Qual a relação entre a maturidade dos repositórios e as suas características de qualidade?
    - *Hipótese:* Espera-se que repositórios mais maduros apresentem árvores de herança mais profundas e maior acoplamento, devido ao acúmulo de código legado e à complexidade arquitetural que cresce naturalmente com o passar dos anos.
- **RQ 03:** Qual a relação entre a atividade dos repositórios e as suas características de qualidade?
    - *Hipótese:* Repositórios com alta frequência de *releases* (alta atividade) podem apresentar uma ligeira queda na coesão (LCOM), refletindo a pressa na entrega contínua de novas *features*, que nem sempre permite refatorações ideais a cada ciclo.

- **RQ 04:** Qual a relação entre o tamanho dos repositórios e as suas características de qualidade?
    - *Hipótese:* Naturalmente, espera-se que repositórios maiores (com mais linhas de código) possuam um acoplamento (CBO) maior, simplesmente pelo aumento absoluto no volume de classes e métodos que precisam interagir entre si no ecossistema do software.

2. Metodologia

Para responder às questões de pesquisa propostas, o experimento foi estruturado nas seguintes etapas metodológicas:



2.1 Seleção dos Repositórios

O escopo da pesquisa consiste na análise dos 1.000 repositórios mais populares escritos na linguagem Java, hospedados no GitHub. Devido ao alto volume de dados, optou-se por construir um *pipeline* automatizado em Python. O *script* realiza consultas paginadas à API GraphQL do GitHub para contornar limites de requisição. Para otimizar o armazenamento local (disco), o *script* adota uma abordagem sob demanda: cada repositório é clonado individualmente, analisado pela ferramenta estática e, imediatamente após a extração das métricas, seu diretório é excluído do sistema.

2.2 Coleta de Métricas de Processo

As informações sobre o processo de desenvolvimento dos repositórios selecionados foram extraídas diretamente dos nós retornados pela API GraphQL. As métricas de processo definidas são:

- **Popularidade:** Medida pelo número de estrelas do repositório.
- **Tamanho:** Medido através de linhas de código e linhas de comentários.
- **Atividade:** Medida pelo número de *releases* lançadas.
- **Maturidade:** Medida pela idade do repositório, em anos.

2.3 Coleta de Métricas de Qualidade

Para a avaliação da qualidade interna do produto, os repositórios selecionados terão seu código-fonte analisado estaticamente utilizando a ferramenta CK. As métricas de qualidade extraídas serão:

CBO (*Coupling Between Objects*): Mede o nível de acoplamento entre as classes.
DIT (*Depth Inheritance Tree*): Mede a profundidade da árvore de herança.
LCOM (*Lack of Cohesion of Methods*): Mede a falta de coesão nos métodos das classes.

2.3 Sumarização e Análise dos Dados

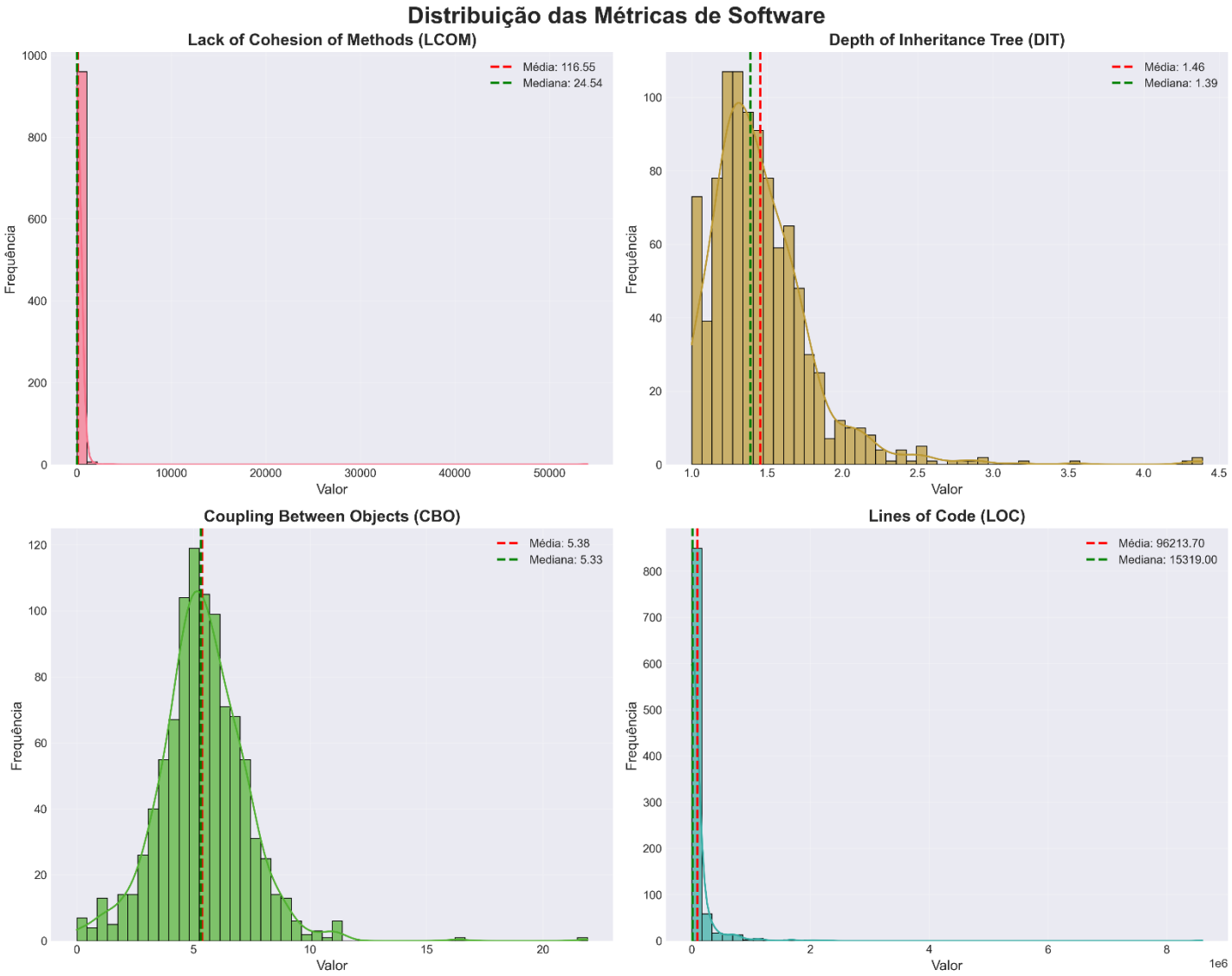
Como a ferramenta CK emite resultados detalhados em diversos níveis (gerando arquivos .csv granulares), os dados de qualidade coletados passarão por uma sumarização por repositório. Para cada questão de pesquisa, a análise será feita comparando as métricas de processo com os valores de medida central (média, mediana e desvio padrão) extraídos das métricas de qualidade.

3. Visualização dos Resultados

Resumo estatístico geral

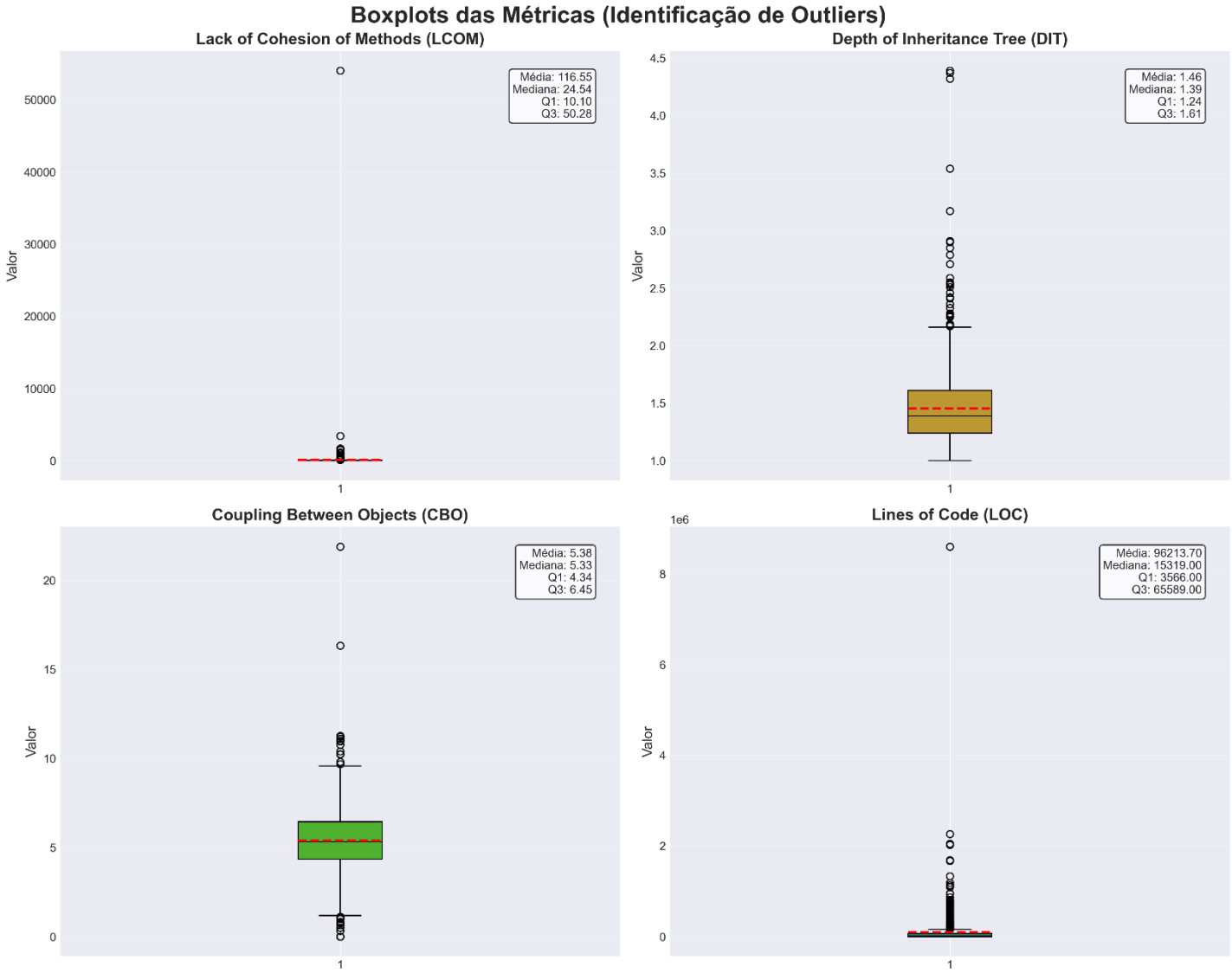
Métrica	Média	Mediana	Desvio Padrão
CBO	5,38	5,33	1,88
DIT	1,46	1,39	0,35
LCOM	116,55	24,54	1743,03
Tamanho (LOC)	96213,70	15319,00	350268,85

Fonte: Autoria própria (2026).



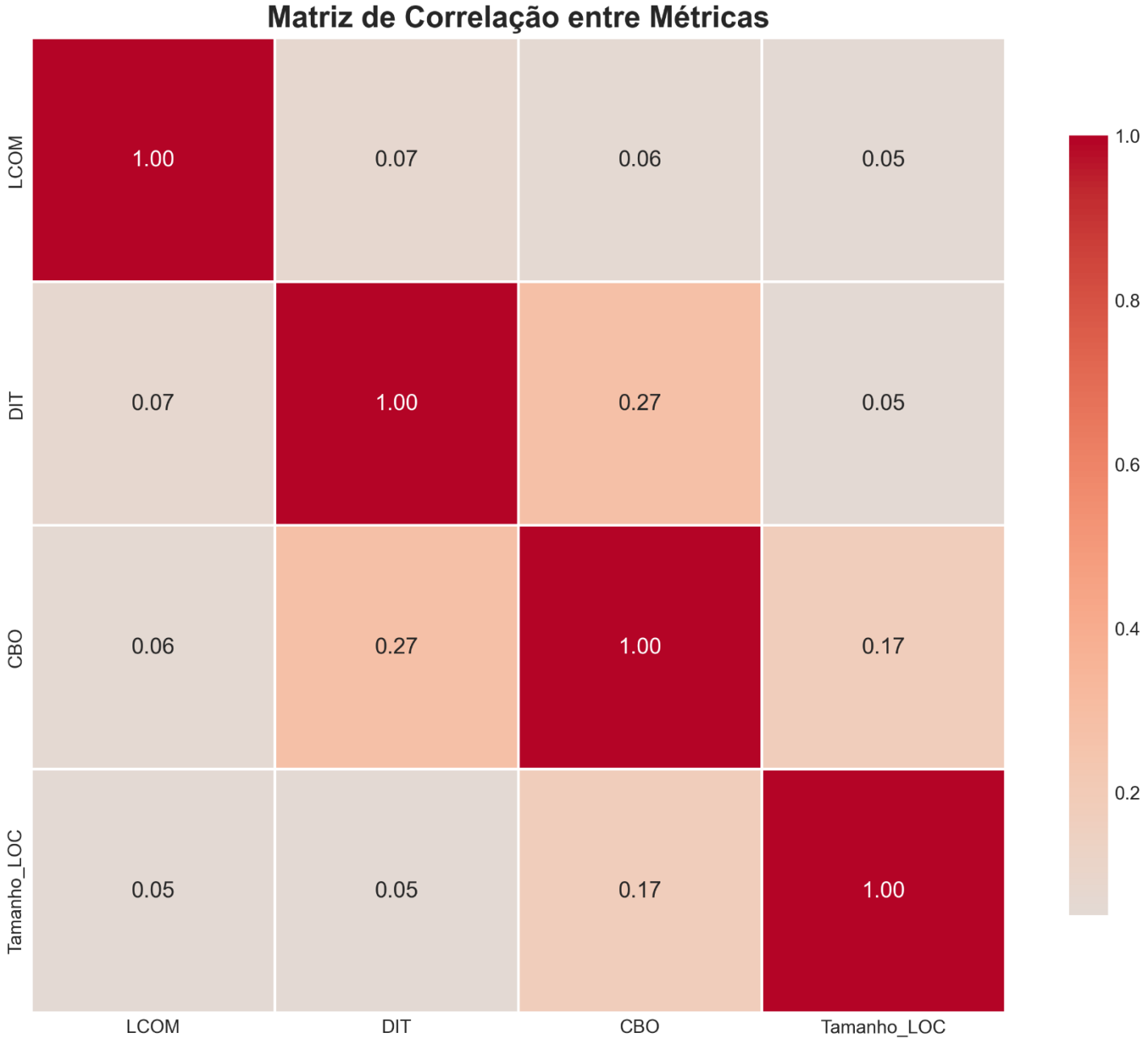
Representação em histogramas das métricas coletadas LCOM, DIT, CBO e LOC.

PUC Minas

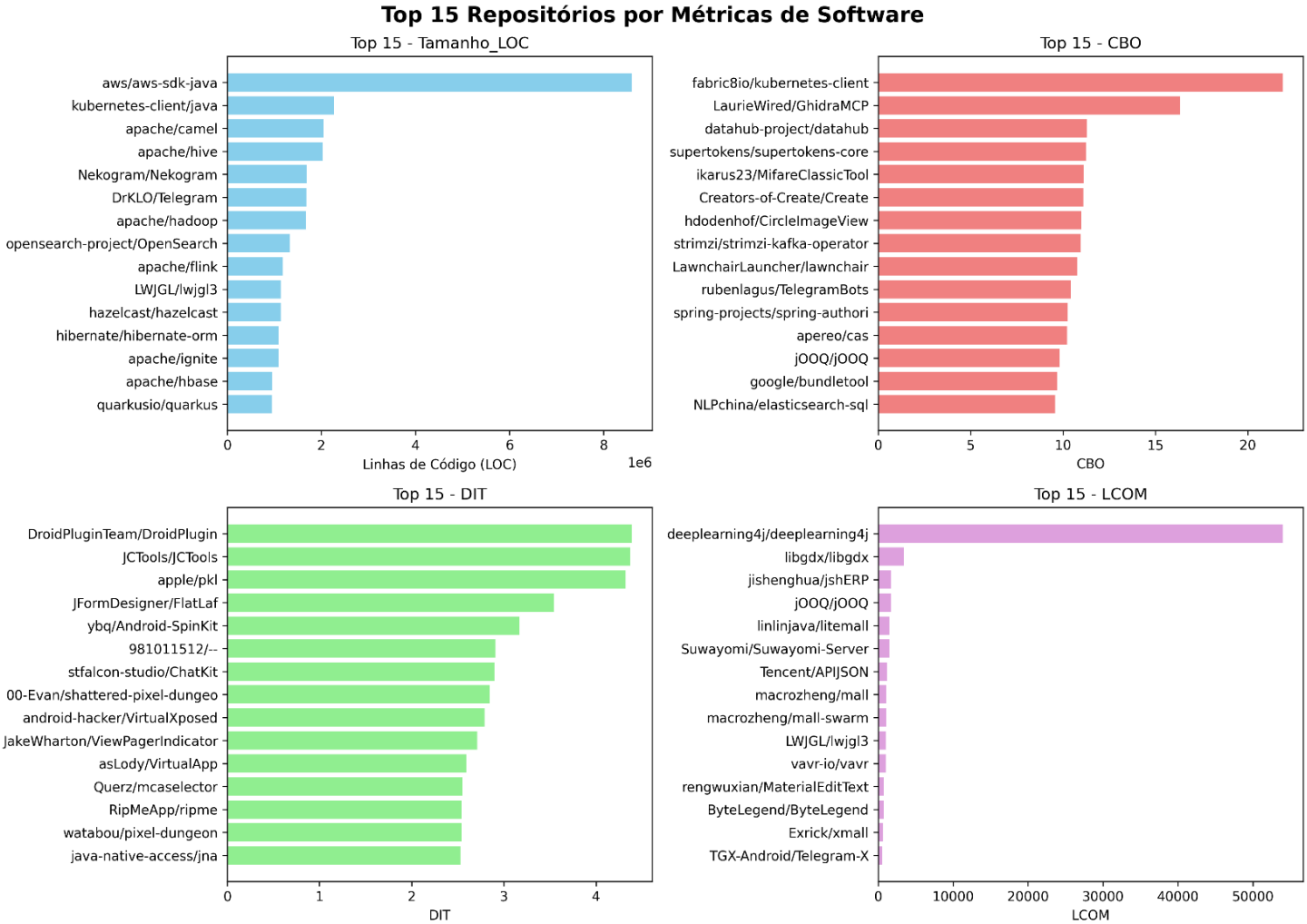


Representação em gráficos de boxsplot das métricas coletadas LCOM, DIT, CBO e LOC.

PUC Minas



Representação em matriz de correlação das métricas coletadas LCOM, DIT, CBO e LOC.



Gráficos de barras empilhadas representados os top 15 repositórios de cada métrica coletada.

4. Conclusão

Este estudo investigou a relação entre métricas de processo e de qualidade interna em 1.000 repositórios Java, revelando um ecossistema de software heterogêneo e marcado por altíssima variância. A análise descritiva evidenciou *outliers* extremos nas métricas de Tamanho (LOC) e Falta de Coesão (LCOM), indicando que a mediana retrata a realidade estrutural dos projetos com muito mais precisão do que a média. Ao cruzar os dados, constatou-se uma correlação linear quase nula (0.17) entre o tamanho do repositório e o seu acoplamento (CBO), refutando a hipótese de que o aumento no volume de código degrada obrigatoriamente a arquitetura (RQ 04).

De forma semelhante, características externas como a popularidade, maturidade e atividade não se mostraram preditores determinísticos para a degradação da qualidade do código (RQ 01, 02 e 03). Os dados demonstram que repositórios maduros, altamente ativos e populares conseguem manter árvores de herança curtas e acoplamento baixo. Conclui-se, portanto, que a saúde interna de um sistema *open-source* depende

primariamente da adoção de práticas rigorosas de engenharia de software pelas equipes mantenedoras como testes automatizados e refatorações contínuas, independentemente da escala geométrica ou do sucesso público que o projeto atinja.